Optimización

El problema del viajante

Índice

Esquema 2

Ideas clave 3

7.1 Introducción 3

7.2 Objetivos 4

7.3 Fundamentos de teoría de grafos 4

7.4 Matriz de Adyacencia - Matriz de Incidencia 9

7.5 Sobre el comportamiento de un algoritmo 12

7.6 Clases de Problemas 13

7.7 Caminos y circuitos eulerianos 15

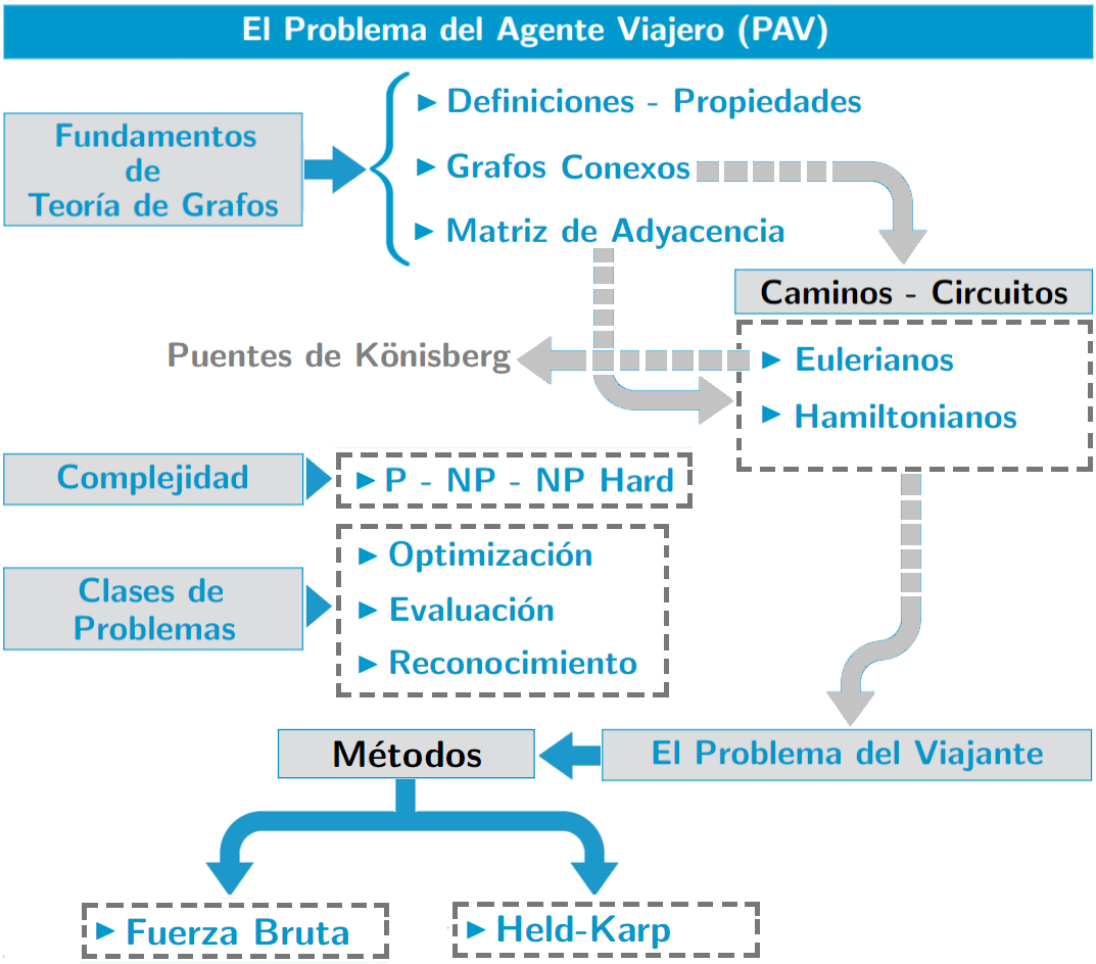
7.8 Caminos y circuitos hamiltonianos 18

7.9 El problema del viajante 21

7.10 Método de Fuerza Bruta 22

7.11 El método de Held-Karp 23

Esquema


El Problema del Agente Viajero (PAV)
Fundamentos de Teoría de Grafos
Definiciones - Propiedades
Grafos Conexos
Matriz de Adyacencia
Caminos - Circuitos
Puentes de Könisberg
Eulerianos
Hamiltonianos
Complejidad
P - NP - NP Hard
Clases de Problemas
Optimización
Evaluación
Reconocimiento
Métodos
El Problema del Viajante
Fuerza Bruta
Held-Karp

Ideas clave

7.1 Introducción

Supongamos que un vendedor (un agente viajero), tiene que planear un viaje que, partiendo y terminando en su propia ciudad, debe recorrer todas las ciudades de una lista dada pero pasando una sola vez por cada una de ellas.

Una condición razonable para este viaje, es planificar la ruta de manera que la distancia total viajada sea la menor posible. Supongamos que el vendedor sabe, para cada par de ciudades dado, la distancia de una a la otra, y en esas condiciones, el viajero tiene los datos necesarios para encontrar el camino que **minimiza** la distancia. Sin embargo, la resolución de este problema de optimización no es, en absoluto, trivial.

El **problema del viajante** (o el problema del agente viajero), es un modelo de importancia superlativa en teoría de optimización. Esta importancia, no se desprende de su aplicación directa (no hay una organización internacional de agentes viajeros pidiendo una solución óptima para este problema), sino más bien por ser un problema marco para la rama de la optimización combinatoria.

La función objetivo del problema del viajante es la distancia total pero resulta evidente que no podemos emplear los métodos que hemos estudiado en este curso ya que el espacio de variables no es un continuo (como hasta ahora), si no que es un espacio de caminos. La modelización del problema es, entonces, un primer escollo que nos encontramos a la hora de abordarlo.

La teoría de grafos es el marco matemático adecuado para plantear el problema del agente viajero. Esta teoría fue desarrollada inicialmente por Leonhard Euler cuando

se enfrentó al problema de los puentes de Königsberg. La primera parte de este tema, se dedicará a dar una introducción a la teoría de grafos. Una vez familiarizados con sus aspectos básicos, procederemos a enunciar el problema del viajante utilizando elementos de teoría de grafos y veremos en consecuencia, estrategias para resolverlo.

7.2 Objetivos

Para este tema se pueden plantear algunos objetivos específicos y que funcionen de forma concatenada. A saber,

1. Estudiar y construir caminos eulerianos y hamiltonianos.
2. Determinar la matriz de adyacencia de un grafo y viceversa, dada la matriz de adyacencia poder construir el grafo. Lo propio con la matriz de Incidencia.
3. Determinar condiciones para caminos específicos como los hamiltonianos, derivados del problema del viajante.
4. Estudiar métodos que nos permitan resolver este tipo de problemas: **Método de Fuerza Bruta** y **Método de Held-Karp**.
5. Implementar estos métodos para poder resolver problemas más complicados.

7.3 Fundamentos de teoría de grafos

Un grafo $G(V, A)$ es un objeto formado por un conjunto de puntos $V = \{v_1, v_2, \ldots\}$ y un conjunto de líneas $A = \{a_1, a_2, \ldots\}$, de manera que cada línea une dos puntos.

Si se impone una orientación en las líneas (esto es, se define un punto de partida y otro de llegada), estas se denominan **arcos** y el grafo se llama **dirigido**.

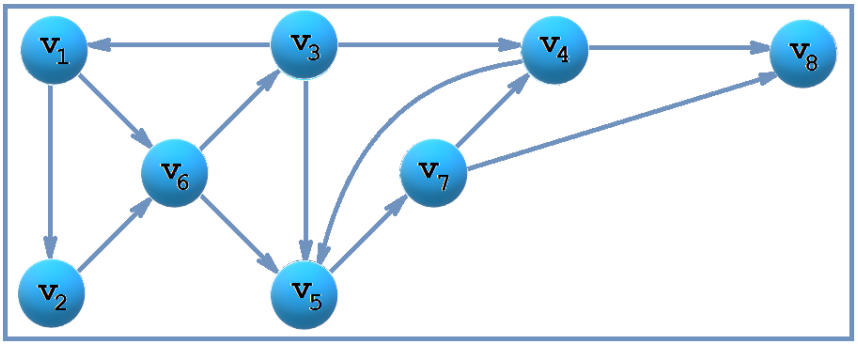


Figura 1: Grafo dirigido con 8 vértices y 13 arcos

En caso de que las líneas no estén orientadas, éstas se denominan **aristas** y el grafo se denominará no dirigido. Los puntos del grafo siempre se denominarán como **vértices**. En este curso, los grafos siempre estarán formados conjuntos de aristas (o arcos) y vértices finitos.

En un grafo dirigido G, los sucesores de un vértice v_i son el conjunto $\Gamma(v_i)$ de todos los vértices a los cuales se puede llegar desde v_i mediante un arco. De manera similar, podemos definir los predecesores $\Gamma^{-1}(v_i)$ como el conjunto de todos los vértices de los cuales se puede llegar a v_i. En general, $\Gamma^n(v_i) = \Gamma(\Gamma^{n-1}(v_i))$. Si G es un grafo no dirigido, entonces no tiene sentido intentar diferenciar los vértices. En consecuencia, se denomina **grado** de un vértice al número de aristas incidentes que tiene.

Un **lazo**, es una arista (o arco) que empieza y termina en un mismo punto y un **camino**, es una secuencia ordenada cualquiera de aristas (o arcos) donde el vértice final de una arista es el vértice inicial de la siguiente.

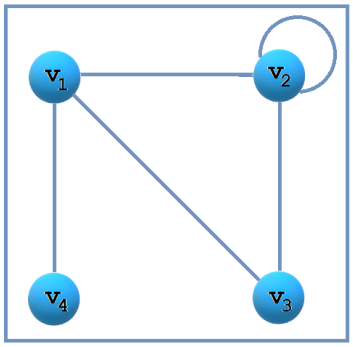


Figura 2: Grafo NO dirigido con un lazo

La distancia $d(v_i, v_j)$ entre los vértices v_i y v_j es el número de aristas que contiene el camino entre los dos vértices con el menor número de ellas. Ahora bien, un camino es **simple** si no repite aristas (arcos) y es **elemental** si no repite vértices.

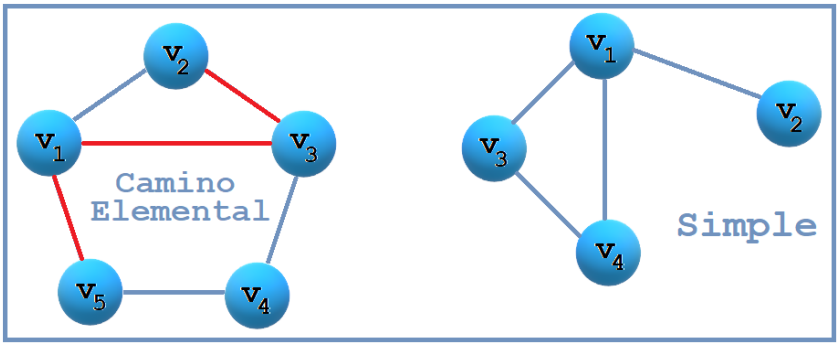


Figura 3: Camino elemental y camino simple

Propiedades fundamentales

- En un grafo no dirigido, todo camino entre dos vértices contiene un camino elemental entre los mismos dos vértices.
- Si un grafo es dirigido, entonces debe cumplirse:

$$\boxed{\sum_{i=1}^{n} |\Gamma(v_i)| + \sum_{i=1}^{n} |\Gamma^{-1}(v_i)| = 2|A|}$$

- Si el grafo es no dirigido, entonces:

$$\boxed{\sum_{i=1}^{n} |\Gamma(v_i)| = 2|A|}$$

Ejemplo 1.

Consideremos el grafo de la **Figura 3** con el camino elemental. Entonces, en este caso tenemos 5 vértices, donde $V = \{v_1, v_2, v_3, v_4, v_5\}$ y 6 aristas en total, en términos de que este grafo es no dirigido. En consecuencia, los sucesores de cada

vértice están dados por:

$$|\Gamma(v_1)| = 3 \ , \ |\Gamma(v_2)| = 2 \ , \ |\Gamma(v_3)| = 3 \ , \ |\Gamma(v_4)| = 2 \ , \ |\Gamma(v_5)| = 2$$

Por consiguiente,

$$\sum_{i=1}^{5} |\Gamma(v_i)| = 3 + 2 + 3 + 2 + 2 = 12 = 2 \cdot 6 = 2|A|$$

El grafo **subyacente** de un grafo dirigido, es el grafo que se obtiene al eliminar la orientación de sus arcos. $G_P(V_P, A_P)$ es un **grafo parcial** de $G(A, V)$ si $A_P \subset A$ y $V_P \subset V$. Un **multigrafo**, es un grafo que contiene aristas (arcos) repetidas.

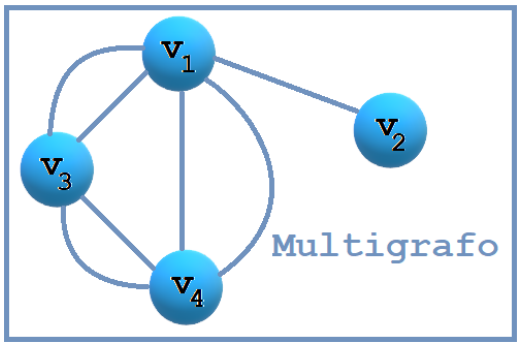


Figura 4: Grafo Multigrafo

Un grafo no dirigido sin lazos es **completo**, si tiene el número máximo posible de aristas.

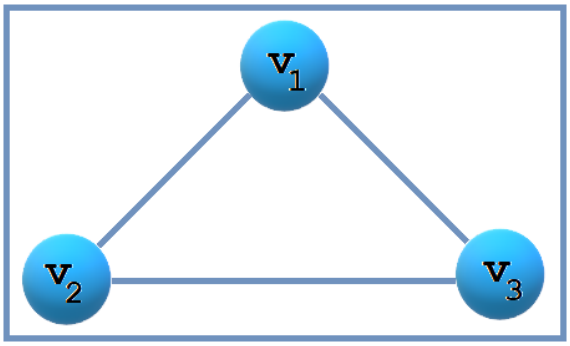


Figura 5: Grafo Completo

Dado un número de vértices r, hay un único grafo completo con ese número de vértices, lo denotaremos por K_r. Podemos entonces definir el grafo **complementario**

$\bar{G}(V, \bar{A})$ de un grafo $G(V, A)$ como el grafo formado por todas las aristas de G que son necesarias para tener un grafo completo.

Grafos Conexos

Un grafo no dirigido es **conexo** si, para cada par de vértices, existe al menos un camino que los une. Cada subgrafo maximal conexo de un grafo G forma una **componente**.

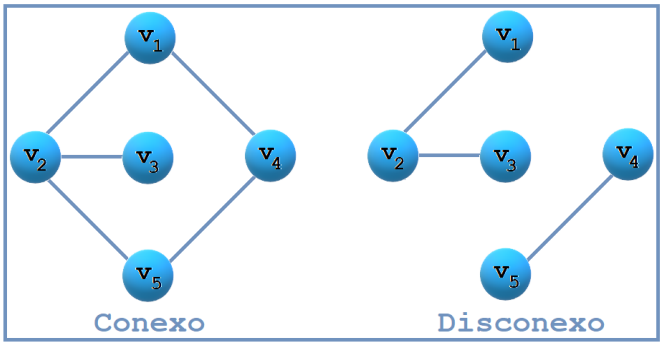


Figura 6: Grafo Conexo vs. Grafo Disconexo

Un **puente**, es una arista que, en caso de ser eliminada, entonces el número de componentes del grafo aumenta. De manera similar, una articulación es un vértice que, en caso de ser eliminado, entonces el número de componentes del grafo aumenta.

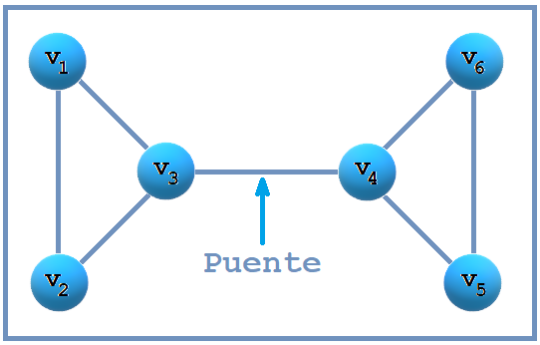


Figura 7: Grafo con un puente

Un grafo no dirigido con dos vértices o más contiene (al menos) dos vértices que no son articulación. Una arista de un grafo no dirigido es un puente, si y sólo si no forma parte de ningún circuito. El grafo **trivial**, es aquel grafo que consta de un sólo vértice y ninguna arista.

Un **árbol**, (no dirigido) es un grafo no dirigido y conexo que no tiene circuitos. En todo árbol existe un camino único entre cualquier pareja de vértices.

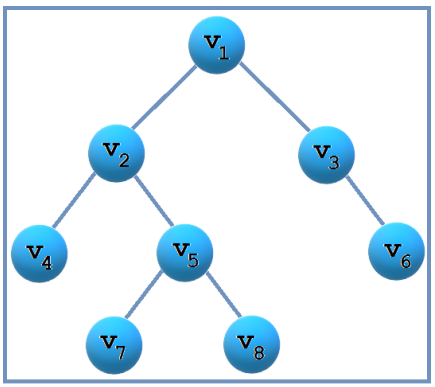


Figura 8: Grafo tipo árbol

Dos grafos son **isomorfos** si existe una correspondencia unívoca entre sus vértices que preserva las adyacencias. Un grafo G_2 es una subdivisión de G_1 si G_2 se obtiene de G_1 mediante el proceso iterativo de substituir una arista a_i de G_1 por un camino donde los únicos vértices comunes con G_1 sean los vértices terminales de a_j, los cuales son los vértices terminales del camino. Se dice también que G_1 y G_2 son **homeomorfos**.

7.4 Matriz de Adyacencia - Matriz de Incidencia

Para poder trabajar con grafos en un ordenador, hace falta representarlos utilizando alguna de las estructuras de información disponibles. En nuestro caso, representaremos un grafo mediante su **matriz de adyacencia.** Dado un grafo G su matriz de adyacencia $A = [a_{ij}]$ es cuadrada y de orden n, si n es el número de vértices del grafo y viene dada por:

$$a_{ij} = \begin{cases} 1 & \text{, si existe una arista entre } v_i \text{ y } v_j, \\ 0 & \text{, en caso contrario.} \end{cases}$$

En resumen, La **matriz de adyacencia** de un grafo, es una matriz booleana que representa las conexiones entre pares de vértices y si un vértice es aislado, entonces la correspondiente fila (columna) esta compuesta sólo por ceros.

Para construir la matriz de adyacencia, se crea una matriz cero, cuyas columnas y filas representan los nodos del grafo. Por cada arista que une a dos nodos, se suma 1 al valor que hay actualmente en la ubicación correspondiente de la matriz.

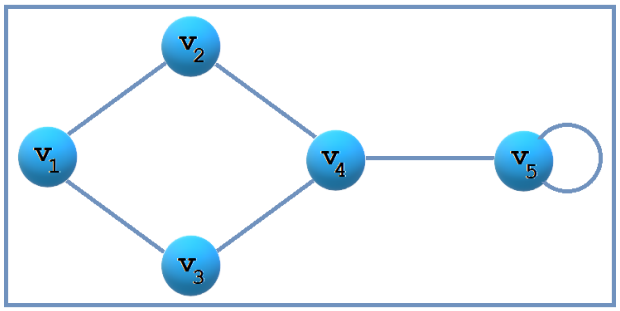
1. Si tal arista es un bucle y el grafo es no dirigido, entonces se suma 1 o 2 (dependiendo de la convención usada).

2. Si el grafo es ponderado, entonces en lugar de un 1 se suma el peso de la arista respectiva.

Es fácil ver que las entradas $a_{i,j}^k$ de las potencias A^k de la matriz de adyacencia representan el número de caminos de orden k (caminos con k aristas) que hay entre los vértices v_i y v_j. Notamos que la matriz de adyacencia es cuadrada y binaria, esto hace que se puedan hacer operaciones algebraicas con ella de manera sencilla y rápida.

Ejemplo 2. Matriz de Adyacencia

Consideremos el grafo no dirigido dado por:



Veamos ahora su matriz de adyacencia:

$$A = \begin{array}{c||ccccc} & v_1 & v_2 & v_3 & v_4 & v_5 \\ \hline\hline v_1 & 0 & 1 & 1 & 0 & 0 \\ v_2 & 1 & 0 & 0 & 1 & 0 \\ v_3 & 1 & 0 & 0 & 1 & 0 \\ v_4 & 0 & 1 & 1 & 0 & 1 \\ v_5 & 0 & 0 & 0 & 1 & 1 \end{array}$$

Nota: Se puede obtener la valencia de cada uno de los vértices, si se suma la fila o columna correspondiente a dicho vértice.

Observaciones:

1. Para un grafo no dirigido, en la matriz de adyacencia la diagonal principal contiene ceros.

2. La representación por medio de matriz se prefiere para algoritmos donde el numero de arcos es grande en proporción al número de vértices.

3. Si sucediera lo contrario se prefiere la representación por medio de listas de adyacencia.

Nota: Esto puede llegar a aprovecharse para ahorrar tiempo en algunos algoritmos.

Volviendo a la matriz de adyacencia del **Ejemplo 2**, tenemos que:

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 & 2 & 0 \\ 0 & 2 & 2 & 0 & 1 \\ 0 & 2 & 2 & 0 & 1 \\ 2 & 0 & 0 & 3 & 1 \\ 0 & 1 & 1 & 1 & 2 \end{bmatrix}$$

En la diagonal de la matriz, se obtiene el grado (o la valencia) de los vértices, y los demás elementos de esta matriz cuadrada indican el número de lados de longitud 2 de v_i a v_j. En efecto,

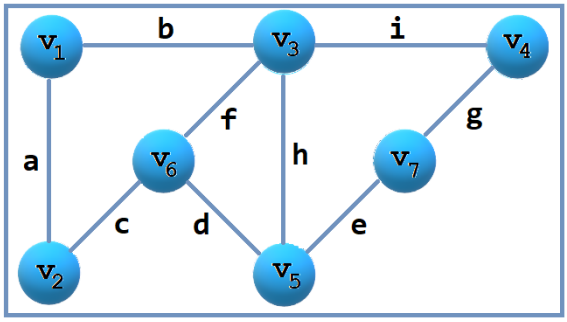
$$\begin{aligned} A^2[1,4] = 2 &\dashrightarrow (v_1,v_2)(v_2,v_4) \text{ y } (v_1,v_3)(v_3,v_4) \\ A^2[2,3] = 2 &\dashrightarrow (v_2,v_1)(v_1,v_3) \text{ y } (v_2,v_4)(v_4,v_3) \\ A^2[2,5] = 1 &\dashrightarrow (v_2,v_4)(v_4,v_5) \end{aligned}$$

Desventajas

- Poco eficiente si el número de vértices varia a lo largo del tiempo de vida del grafo.
- Puede darse el caso que el numero de vértices sea mayor del número inicial.
- Poco eficiente cuando el grafo tiene poco arcos (la matriz es dispersa).

Matriz de Incidencia

A diferencia de la matriz de adyacencia, la matriz de **incidencia** establece la conexión de aristas con vértices. A saber, las columnas de la matriz representan a las aristas y las filas a los vértices. En este sentido, no pueden haber más de dos unos en cada columna y además, esta matriz no necesariamente es cuadrada. Para visualizar este hecho, consideremos el siguiente grafo y al lado su matriz de incidencia



	a	b	c	d	e	f	g	h	i
v_1	1	1	0	0	0	0	0	0	0
v_2	1	0	1	0	0	0	0	0	0
v_3	0	1	0	0	0	1	0	1	1
v_4	0	0	0	0	0	0	1	0	1
v_5	0	0	0	1	1	0	0	1	0
v_6	0	0	1	1	0	1	0	0	0
v_7	0	0	0	0	1	0	1	0	0

7.5 Sobre el comportamiento de un algoritmo

Para resolver el problema del viajante más adelante, tendremos que utilizar distintos algoritmos. Es conveniente, pues, establecer cierta nomenclatura y resultados generales que nos permitan valorar la eficiencia de estos métodos. El objetivo fundamental en el análisis de un algoritmo, es poder estimar el tiempo de ejecución.

La complejidad de un algoritmo será determinada por el número de pasos que hay que realizar para convertir los datos iniciales al resultado final. Para evaluar la complejidad de un algoritmo, es habitual descartar los factores constantes y estimar el comportamiento cuando el volumen N de los datos de entrada tiende a infinito. Evidentemente, distintos datos de entrada en un algoritmo pueden tener distintos comportamientos.

Por ejemplo, puede que la solución se dé en pocos pasos y para evitar un sesgo de estas características, siempre se evalúa el comportamiento de un algoritmo pensando en el caso más desfavorable.

Diremos que una función $g(N)$ es $\mathcal{O}(f(N))$ de $f(N)$, si existen constantes a y b tales que, para todo $b \geq N$ se tiene que $g(N) \leq af(N)$. Se suele decir que $g(N)$ es del orden de $f(N)$. En cada algoritmo hay que determinar qué operaciones fundamentales se realizan para poder evaluar la complejidad expresada como el número total de operaciones en relación al volumen de los datos de entrada. Habitualmente, $\mathcal{O}(1)$ denota una constante. Si $f(N)$ es $\mathcal{O}(s(N))$ y $g(N)$ es $\mathcal{O}(r(N)$, entonces:

$$f(N) + g(N) = \mathcal{O}(s(N) + r(N)),$$

$$f(N)g(N) = \mathcal{O}(s(N)r(N)).$$

En general, esta relación no es cierto en caso de que las operaciones sean la resta o la división.

7.6 Clases de Problemas

Un problema de **optimización combinatoria**, se define como un conjunto de casos $\mathcal{C}$ tal que cada caso es una pareja $(\mathcal{F}, c)$ donde:

1. $\mathcal{F}$ es el conjunto de las soluciones factibles del problema.
2. $c : \mathcal{F} \longrightarrow \mathbb{R}$ una función de costo.

El objetivo es encontrar el caso que minimiza (o maximiza) $c(\mathcal{F})$.

Hay tres tipos de formas de plantear un problema de optimización combinatoria:

1. **Optimización**: Buscar la (una) solución óptima.

2. **Evaluación**: Busca el costo de la solución óptima.

3. **Reconocimiento**: Dado un caso $(\mathcal{F}, c)$ y un entero r, se responde sí o no a la pregunta: "¿Existe una solución factible $f \in \mathcal{F}$ tal que $c(f) \leq r$"

Para poder formalizar los resultados sobre complejidad, hace falta un modelo de máquina computacional que pueda ejecutar todo tipo de algoritmos. Hay modelos de este tipo (aunque todos ellos son equivalentes). El más popular de todos ellos es la **máquina de Turing** (MT). Una MT, está formada por una unidad de control que se puede encontrar en un número finito de estados E y por una unidad de almacenamiento, en forma de cinta de lectura/escritura. La cinta contiene una secuencia de celdas y cada celda, contiene un símbolo de un alfabeto finito A. La unidad de control permita mover la cinta hacia izquierda o derecha, celda a celda. Finalmente, existe una tabla en la cual se indica como cambia el estado de la unidad de control, qué escribe en la cinta y hacia donde se mueve.

A partir de la información contenida en las n primeras celdas, la MT opera hasta que llega a un estado de parada final. El resultado del proceso queda escrito en la cinta. Según si el estado final es aceptado o no (bajo algún criterio fijado) se dice que la MT acepta o no el conjunto de datos. Llamamos a un conjunto inicial de datos aceptados, casos favorables. El conjunto de todos los casos favorables forma el lenguaje reconocido por la MT.

Si para cada estado, existe otro estado resultado de mover la cinta hacia la izquierda, la derecha (o no moverla), se dice que la MT es determinista (MTD). En caso contrario, la MT es no determinista (MTND). Una propiedad importante de las MTND es que se pueden ver como generalizaciones de las MTD. En efecto, una MTND es una MTD en la que se permite el procesamiento en paralelo de distintas alternativas.

La complejidad temporal de una MTD en función de la longitud n de los datos de entrada viene dada por el número máximo de iteraciones que puede llegar a necesitar

la máquina hasta detenerse para cualquier entrada de longitud n. Para el caso de las MTND, la complejidad temporal viene dada por el número máximo de iteraciones que puede llegar a necesitar para detenerse para cualquier entrada de longitud n, siendo el tiempo que la alternativa más corta.

Utilizando el concepto de MT, se pueden clasificar los problemas en función de su complejidad temporal de la siguiente manera:

- El problema pertenece a la clase de problemas P (polinomial) si el conjunto de casos que tienen respuesta favorable coincide con el lenguaje aceptado por una MTD que tiene complejidad temporal polinomial respecto a la longitud de los datos de entrada.
- El problema pertenece a la clase de problemas NP (polinomial no determinista) si el conjunto de casos favorables coincide con el lenguaje aceptado por una MTND que tiene complejidad temporal polinomial respecto a la longitud de los datos de entrada.

Como las máquinas MTND, generalizan las MTD, entonces se deduce directamente que $P \subset NP$. La igualdad de conjuntos es un problema abierto. Diremos que un problema de reconocimento se puede transformar polinomialmente en otro problema de reconocimiento si dada una entrada del problema original podemos construir una entrada del segundo problema en tiempo polinomial tal que la entrada construida tiene respuesta sí en el segundo problema, si y sólo si, la entrada original tiene respuesta sí en el problema original. Un problema se llama NPC si es NP y todos los demás problemas NP se pueden reducir a él polinomialmente.

7.7 Caminos y circuitos eulerianos

Sea $G(V, A)$ un grafo no dirigido y conexo (o dirigido con grafo subyaciente conexo). Un **circuito euleriano**, es un camino que pasa por cada arista exactamente una vez y

donde se pueden repetir vértices.

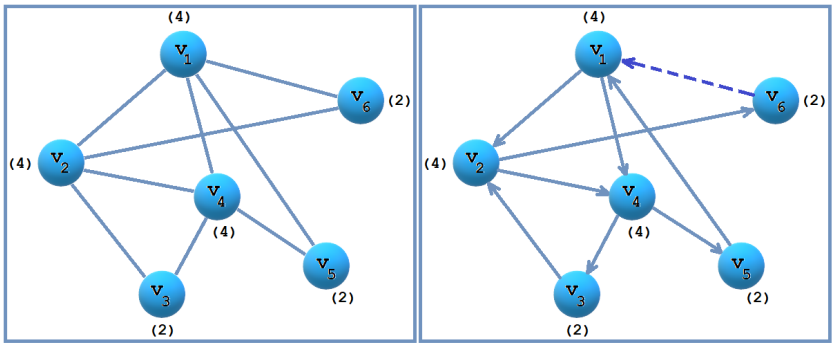


Figura 9: Grafo conexo con valencias pares en sus vértices

Nota: Para el grafo anterior, un camino es justamente:

$$\boxed{6} - 1 - 2 - 4 - 5 - 1 - 4 - 3 - 2 - \boxed{6}$$

y en efecto, podemos conseguir otro:

$$\boxed{4} - 3 - 2 - 4 - 5 - 1 - 2 - 6 - 1 - \boxed{4}$$

donde estos caminos no repiten aristas, pero terminan donde se inicia.

Entonces, resulta necesario hallar un camino que recorra todas las vías (aristas) iniciando en un vértice y terminando en otro distinto. Observemos que del hecho de tener grado par en todos los vértices, resulta muy complicado tratar de hallar este camino, en función de que no nos conduce a un vértice distinto a la partida.

Considere ahora el mismo grafo, pero agregando la arista que conecta a los vértices v_5 y v_6. Como observamos en la siguiente figura:

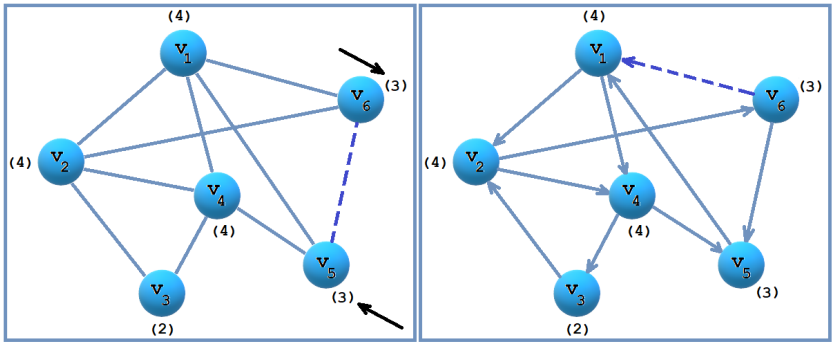


Figura 10: Grafo conexo dos vértices con valencia impar

Observemos que en este caso, un camino euleriano puede ser:

$$\boxed{6} - 1 - 2 - 4 - 5 - 1 - 4 - 3 - 2 - 6 - \boxed{5}$$

Ahora, intentemos el camino:

$$\boxed{5} - 4 - 3 - 2 - 4 - 1 - 6 - 2 - 1 - 5 - \boxed{6}$$

Por tanto, esa imparidad tanto en el vértice 5 como en el 6, ya que ambos eran pares, permitió entrar y salir sin quebrantar la regla de la repitencia de aristas. ¡Pareciera una especie de condición!, no?

Condición para la existencia

Si existen al menos dos vértices de **grado impar**, podemos asegurar que existe un camino euleriano que además inicia en uno de ellos y termina en el otro. Teniendo en cuenta que partimos de un vértice inicial v_i y llegamos a un vértice final v_f, donde $v_i \neq v_f$.

Por ejemplo, consideremos el grafo de la sigueinte figura:

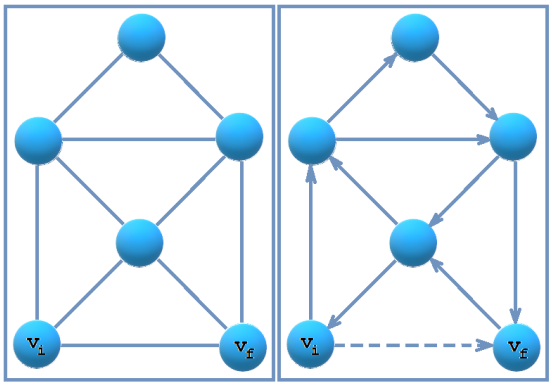


Figura 11: Grafo conexo con camino euleriano

Un **ciclo euleriano - circuito euleriano**, es un camino euleriano que se cierra del todo, es decir, $v_i = v_f$. Por tanto, todos los vértices deben tener grado par. En este sentido, el grafo de la **Figura 9** representa un grafo con circuitos eulerianos.

7.8 Caminos y circuitos hamiltonianos

Sea $G(V, A)$ un grafo no dirigido y conexo (o dirigido con grafo subyaciente conexo). Se define un **camino hamiltoniano**, como un camino simple (que no repite vértices) y que incluye todos los vértices de G. Ahora, un **circuito hamiltoniano** es un camino que pasa por cada vértice exactamente una vez.

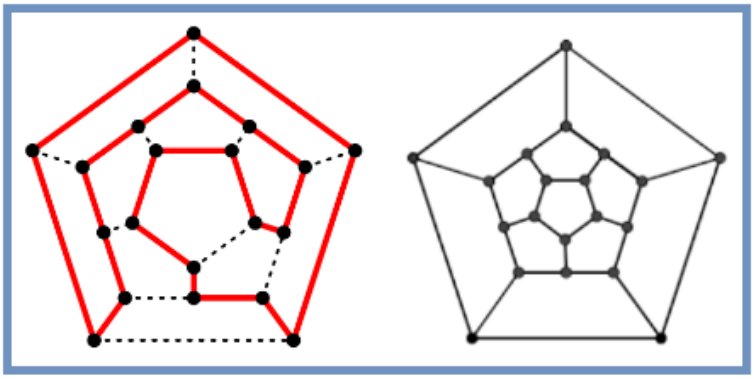


Figura 12: Camino hamiltoniano vs. Circuito hamiltoniano

Establecer si un grafo es hamiltoniano es un problema NPC. Además, no se conocen condiciones que caractericen la existencia de caminos hamiltonianos. Con lo cual, los resultados sobre la existencia de caminos hamiltonianos son necesariamente parciales y los algoritmos que busquen una solución exacta serán, en la práctica, viables el conjunto de vértices del grafo no es demasiado grande. En este contexto, los algoritmos que buscan soluciones aproximadas pasan a ser más relevantes. El siguiente resultado es una condición necesaria para la existencia de un circuito hamiltoniano para un grafo plano (i.e. contenido en un espacio de dos dimensiones).

Teorema 1: Existencia de un circuito hamiltoniano

Sea G un grafo plano que contiene un circuito hamiltoniano H y sean R_j y R'_j el numero de regiones de G que están dentro y fuera de H respectivamente y que están delimitadas por j aristas. Entonces:

$$\sum_{j>2} (j-2)(R_j - R'_j) = 0.$$

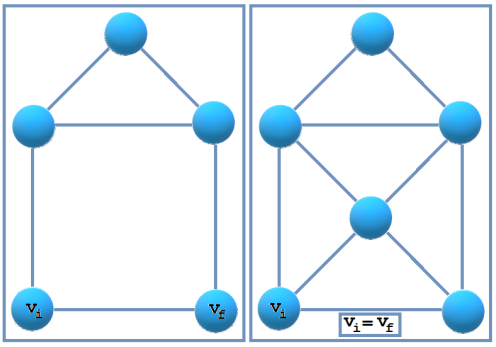
$\square$

En la práctica, este resultado se puede utilizar para demostrar que un grafo plano no contiene un circuito hamiltoniano comprobando que no es posible verificar la relación que impone el teorema.

Los grafos no dirigidos que son completos contienen un circuito hamiltoniano. Por otro lado, los grafos dirigidos cuyos grafos subyacentes son completos, contienen un camino hamiltoniano. Si en un grafo dirigido hay, para toda pareja de vértices v_i y v_j, una arista que une a v_i con v_j y otra que une v_j con v_i, y su grafo subyacente es completo, entonces también existe un circuito hamiltoniano.

Ejemplo 3. Camino - Circuito hamiltoniano

Observe los grafos de la figura:



El de la izquierda representa un camino hamiltoniano, ya que $v_i \neq v_f$. Más aún, no es necesario recorrer todas las aristas. El de la derecha, representa un circuito hamiltoniano, esto en función de que donde inicia termina, por ejemplo si numeramos en sentido horario, el circuito $1-2-3-4-5-6-1$ es hamiltoniano.

¿Cómo determinar si un grafo admite un ciclo hamiltoniano?

Teorema 2: L. Rédei

Si G es un grafo de n vértices, con $n \geq 2$ y u y v son dos vértices cualesquiera, entonces G admite un circuito hamiltoniano si se verifica que:

$$\boxed{\text{grado}(u) + \text{grado}(v) \geq n - 1}$$

Volviendo al **Ejemplo 3**, tenemos que: $n = 5$ y hay dos vértices de grado 3 y el resto de grado 2. Al sumar cualquiera de ellos, la suma es mayor o igual a $4 = n - 1$.

Vamos a ver un método para, dado un grafo dirigido de n vértices, encontrar un camino (o circuito) hamiltoniano. Este método se basa en el *backtracking*, es decir, se ensaya una solución y, si esta no funciona, se reconstruye una alternativa empezando desde el final. Para tal fin, vamos a considerar la tabla P indexada por los vértices del grafo en las columnas. Dado una columna identificada por el vértice v, la tabla se completa con los conjuntos $\Gamma(v)$ debajo del vértice v. El camino (o circuito) hamiltoniano se codificará en una lista ordenada S. El algoritmo consta de los siguientes pasos:

1. Se escoge un vértice inicial v_i. Utilizando la tabla P, el primer vértice v_i de la columna correspondiente a v_1 se añade a la lista S. A continuación, el primer vértice posible (no utilizado anteriormente) v_j de columna correspondiente a v_i se incorpora a S. Así sucesivamente.

2. Supongamos que el estado de nuestra lista es $S = \{v_1, v_i, v_j, , \ldots v_r\}$. Hay dos maneras de bloquear la incorporación de un nuevo vértice a S:

   - No hay ningún vértice posible en la columna de v_r,
   - el camino representado por S tiene $n-1$ arcos. Entonces ya hemos terminado.

   En caso de que se de (b) y queremos un circuito hamiltoniano (no un camino), hay que comprobar si existe el arco (v_r, v_1). Si tal es el caso, se habrá llegado a una solución. Si no existe tal arco, entonces hay que hacer un *backtrack*. Si se da el caso (a), entonces habrá que hacer un *backtrack* directamente.

   El *backtrack* consiste en eliminar de la lista S el último vértice añadido y substituirlo por el siguiente vértice posible en la columna correspondiente a v_{r-1}. Si tal vértice no existe, haremos el *backtrack* desde el vértice v_{r-1}. Finalmente el proceso terminará si se llega a v_1 y no hay nuevos vértices disponibles para continuar el proceso.

7.9 El problema del viajante

Retomamos ahora el problema del viajante, ahora que tenemos las herramientas matemáticas necesarias para formularlo y encararlo con garantías. Recordamos que, en la introducción de este tema, hemos descrito el problema del viajante como encontrar un viaje que, partiendo de y terminando en su propia ciudad, recorrer todas las ciudades de una lista dada pero pasando una sola vez por cada una de ellas y que la distancia sea mínima.

Adaptando este enunciado en términos de la teoría de grafos. En primer lugar debemos tener en cuenta que enunciaremos el problema en términos de optimización combinatoria. Para tal fin, se necesita una función de costo. En general, cualquier arista que una los vértices v_i y v_j se puede cargar con un costo c_{ij}.

En el caso del problema del viajante, tiene sentido que el costo de cada arista represente el coste del viaje de una ciudad a otra. Hay varias formas de cuantificar el coste del desplazamiento: Una opción razonable es la distancia entre las ciudades pero también se puede elegir el tiempo de transporte o la cantidad de combustible. En un sentido más abstracto, se pueden escoger distintas métricas y eso hace que el problema adquiera ciertas particularidades.

El problema del viajante se formula, a través de la teoría de grafos, de la siguiente manera:

> Dado un grafo (dirigido o no) $G(V, A)$ con coste c_{ij} asociado a la arista (o arco) a_{ij}, se trata de encontrar un circuito hamiltoniano cuyo costo asociado sea mínimo.

La condición de que la ciudad de inicio y la final sea la misma, se puede eliminar, en este caso el problema trataría de encontrar un camino hamiltoniano. Por otro lado, también es común considerar el problema del viajante generalizado, que elimina el requisito de que cada ciudad se visite una sola vez. En este caso, ya no hay necesidad de buscar un circuito (o camino) hamiltoniano ya que repetir vértices está permitido.

Dada la alta complejidad computacional del problema del viajante, existen varias estrategias distintas para resolverlo (según las necesidades concretas del problema

real que se esté modelando). Los algoritmos exactos solo funcionan para problemas de tamaño muy reducido. Otra estrategia es encontrar soluciones sub-óptimas mediante algoritmos heurísticos. En este tema, discutiremos algoritmos que busquen soluciones exactas mientras que, en los siguientes temas, discutiremos cómo abordar el problema del viajante mediante algoritmos heurísticos.

A partir de ahora, vamos a suponer que nuestro grafo es completo. Esta hipótesis extra se verifica en la mayoría de situaciones prácticas. Además, si un grafo no es completo, se le pueden añadir aristas artificiales con un costo asociado infinito (o muy grande) de manera que cualquier algoritmo descarte soluciones que contengan esas aristas. Suponiendo, pues, que el grafo es completo, no resta generalidad al modelo.

7.10 Método de Fuerza Bruta

Éste es el menos aconsejable de todos y el primero en que uno piensa cuando se enfrenta al problema. El método de la **fuerza bruta**, consiste en calcular todos los circuitos (o caminos) hamiltonianos y calcular las distancias totales para cada uno de ellos. Luego, se escoge el que tenga la distancia mínima.

Hay que tener en cuenta que un grafo completo de n vértices, contiene $n!/2$ caminos hamiltonianos distintos y $(n-1)!/2$ circuitos hamiltonianos distintos. El algoritmo, pues, se divide en los siguientes pasos:

- Calcular todos las soluciones válidas,
- Calcular la distancia de cada solución encontrada,
- Escoger la solución que tenga la menor distancia.

Este algoritmo básico propuesto para el PAV, consiste en efecto, en generar todas las permutaciones posibles de los vértices dados y ver cuál es el camino de mínimo coste que une todos ellos. Para esto se siguen los siguientes pasos:

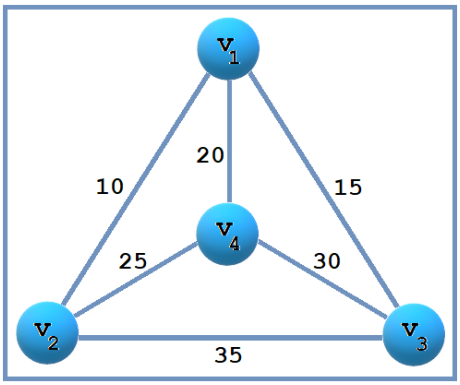
- Se denota como ciudad 1 aquella que sea el punto de inicio y final del camino.
- Se generan todas las $(n-1)!$ permutaciones de las ciudades.

- Se calcula el coste de todas las permutaciones y se elige el camino que corresponde a la permutación de mínimo coste.
- Se vuelve a la ciudad 1 siguiendo el camino elegido.

El tiempo computacional para este algoritmo es $\mathcal{O}(n!)$. Es un tiempo computacional muy grande, por lo que no es un algoritmo útil.

Ejemplo 4. Método de Fuerza Bruta

onsidere el grafo completo dado por:



Determine el camino que realiza el mínimo. Para ello, agruparemos los datos en la siguiente tabla:

Algoritmo Fuerza Bruta	
Permutaciones	**Coste**
$1-2-3-4-1$	95
$1-2-4-3-1$	80
$1-3-2-4-1$	95
$1-3-4-2-1$	80
$1-4-3-2-1$	95
$1-4-2-3-1$	95

7.11 El método de Held-Karp

Ya hemos explorado la solución del problema del viajante utilizando fuerza bruta y ya hemos visto que, en general, es una mala estrategia. Ahora exploramos un método (el de **Held-Karp**) que también ofrece una solución exacta del problema, pero con un mejor comportamiento.

Antes de seguir, definimos un elemento que será esencial para la construcción del algoritmo, la matriz de distancias:

Para un problema del viajero con cargas $c_{i,j}$ en las aristas, definimos la matriz $D = [d_{i,j}]$ como:

$$d_{i,j} = \text{mín}\{c_{i,j}\}.$$

Aquí estamos asumiendo varios hechos. En primer lugar, admitimos que pueda haber varias aristas con distintas distancias uniendo los mismos vértices. Esto se hace porque no implica ningún gran cambio en el planteamiento del algoritmo. Se escoge la distancia menor de todas las posibles. También recordamos que estamos suponiendo que el grafo es completo, es decir, que para cada i y j, existe una arista (arco) que une el vértice v_i con el v_j. Recordamos que, si en la práctica no es así, añadimos artistas artificiales con costo infinito. Esto se deberá reflejar en la matriz de distancias, pues si entre dos vértices v_i y v_j hemos tenido que añadir una arista artificial, entonces $d_{i,j} = \infty$. Notamos que, si el grafo es no dirigido, entonces la matriz D es simétrica.

Una premisa importante para plantear el algoritmo de Held-Karp, es el ciclo (camino) hamiltoniano óptimo no depende del vértice que se etiqueta inicialmente. Esto implica que se puede escoger un vértice inicial v_1 cualquiera y fijarlo como vértice inicial del algoritmo sin ninguna pérdida de generalidad.

Ahora bien, vamos a considerar caminos en el grafo. Estos se pueden codificar como conjuntos $S \subset V \setminus \{v_1\}$, donde V es el conjunto de vértices del grafo. No estamos considerando el vértice v_1 en nuestros caminos ya que consideramos que todos ellos empezarán en el mismo. Dado $v_i \in S$, definimos $D(S, v_i)$ como la longitud del camino más corto desde v_1 hasta v_i que pasa por todos los vértices de S una sola vez. Esta definición es extremadamente útil ya que cumple la siguiente formula recursiva:

$$D(S, v_i) = \min_{v_j}(D(S \setminus \{v_i\}, v_j) + d_{j,i}.$$

Si S consta de un solo vértice v_i, definimos $D(S, v_i) = d_{1,i}$. Utilizando la fórmula recursiva anterior, podemos construir una función de costo para subconjuntos de vértices S desde orden 1 hasta orden $n - 1$.

Cuando el algoritmo alcanza el subconjunto S de tamaño máximo, $S = V \setminus \{v_1\}$,

tenemos que el costo del ciclo hamiltoniano óptimo es:

$$\min_{v_i}(D(V \setminus \{v_1\}, v_i) + d_{i,1}).$$

Notamos que el algoritmo de Held-Karp mejora de forma notable la opción de la fuerza bruta. En este algoritmo tenemos que construir los subconjuntos de $V \setminus \{v_1\}$, esto son 2^{n-1} subconjuntos. Para cada uno de ellos, calcularemos la función de costo de cada uno de sus elementos (como mucho hay $n-1$) y para evaluar la función de costo uno necesita calcular una cantidad inferior a n valores basados en los pasos anteriores. La complejidad de este método es, pues, $\mathcal{O}(n^2 2^n)$.

Algoritmo de Held-Karp

Paso 1

Se escoge un vértice inicial, da igual cual. Supongamos que es v_1. Consideramos los subconjuntos $S \subset \{2, \ldots, n\}$ de ciudades. Cada uno de estos subconjuntos codifica una lista de vértices y a la postre, un camino que recorre cada uno de estos vértices una sola vez. Para cada $c \in S$, se puede calcular la distancia mínima del camino que empieza en el vértice v_1 y termina en el vértice c. Denotamos esta distancia mínima por $D(S, c)$.

Paso 2

Si $S = \{c\}$, entonces $D(S, c) = d_{1,c}$. En otro caso,

$$D(S, c) = \min_{x \in S \setminus \{c\}}(D(S \setminus \{c\}, x) + d_{x,c}).$$

Paso 3

La distancia mínima de un ciclo (camino) Hamiltoniano que pase por todas los vértices es:

$$M = \min_{c \in \{2,\ldots,n\}}(D(\{2, \ldots, n\}, c) + d_{c,1}).$$

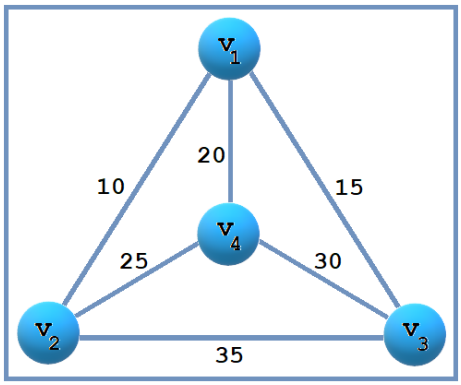
Paso 4

El camino etiquetado por $w_1, \ldots, w_n$ es óptimo si

$$M = D(\{2, \ldots, n\}, w_n) + d_{w_n,1}.$$

Ejemplo 5. Método de Held-Karp

Volvamos al ejemplo anterior:



Matriz de Adyacencia

	v_1	v_2	v_3	v_4
v_1	0	10	15	20
v_2	10	0	35	25
v_3	15	35	0	30
v_4	20	25	30	0

Observaciones:

1. Representamos a S - el subconjunto del gráfico que aún no se ha recorrido. Recordemos que $d_{i,1}$:= Distancia de v_1 a v_i. Consideremos el nodo inicial y final como v_1.

2. En adelante denotaremos a los vértices como $1, 2, 3, 4$, para simplificar notación.

Distancias de v_1 a los nodos adyacentes.

$D(2, \emptyset) = 10$	$D(3, \emptyset) = 15$	$D(4, \emptyset) = 20$

Cálculo de costes (distancias) a nodos adyacentes:

$D(2,\{3\}) = d_{2,3} + D(3,\emptyset) = 50$
$D(2,\{4\}) = d_{2,4} + D(4,\emptyset) = 45$
$D(3,\{2\}) = d_{3,2} + D(2,\emptyset) = 45$
$D(3,\{4\}) = d_{3,4} + D(4,\emptyset) = 50$
$D(4,\{2\}) = d_{4,2} + D(2,\emptyset) = 35$
$D(4,\{3\}) = d_{4,3} + D(3,\emptyset) = 45$

Cálculo de costes (distancias) al cuarto nodo y nodos adyacentes:

$D(2,\{3,4\}) = \text{mín}(d_{2,3} + D(3,\{4\}), d_{2,4} + D(4,\{3\})) = \text{mín}(85, 80) = 80$
$D(3,\{2,4\}) = \text{mín}(d_{3,2} + D(2,\{4\}), d_{3,4} + D(4,\{2\})) = \text{mín}(80, 65) = 65$
$D(4,\{2,3\}) = \text{mín}(d_{4,2} + D(2,\{3\}), d_{4,3} + D(3,\{2\})) = \text{mín}(75, 75) = 75$

Cálculo de costes del nodo inicial al resto:

$$D(1,\{2,3,4\}) = \text{mín}(d_{1,2} + D(2,\{3,4\}), d_{1,3} + D(3,\{2,4\}), d_{1,4} + D(4,\{2,3\})) = \text{mín}(90, 80, 95) = 80$$